TP 3

M Billaud - m2103

19-02-2021

Table des matières

1 Présentation **1**

2 Description des classes **2**
2.1 La classe `Robot` . . . 2
2.2 La classe `Screen` . . . 2
2.3 Interface `Command` . . . 2
2.4 Classe `DrawRightCommand` (exemple de commande) . . . 2

3 Annexes **2**
3.1 Classe `Robot` . . . 2
3.2 Classe `Screen` . . . 3
3.3 Classe `DrawRightCommand` . . . 4

Contenu de la séance : construire un programme à partir de d'indications, de bouts de code fournis, d'explications et de spécifications plus ou moins formelles.

1 Présentation

Objectif : un programme qui simule un "robot" qui effectue un dessin sous le contrôle d'un programme (suite de *commandes*). Par exemple le code

```
static void test() [
    Robot r2d2 = new Robot(20, 10);

    Command[] program = {
        moveTo(1, 5), drawRight(3), drawUp(2),
        drawRight(2), drawUp(2),
        moveTo(6, 5), drawRight(2), drawUp(3)
    };

    r2d2.run(program);
}
```

affichera ce véritable chef d'oeuvre :

```
+----------+
|          |
|      *   |
|      * * |
|    *** * |
|    *   * |
| **** *** |
+----------+
```

NB : `moveTo` et ses congénères sont des méthodes utilitaires qui simplifient la construction des commandes :

```
static Command moveTo(int row, int col) {
    return new MoveToCommand(row, col);
}
```

À faire

1. **Réaliser** un programme qui fonctionne correctement à partir de ce qui est fourni.

Conseil : commencer avec un "programme" qui ne contient qu'une commande "`drawRight`".

2. **Refactoring** : on peut ramener les 4 classes (traits dans les 4 directions) en une seule. Par exemple l'appel de `drawUp(3)` se traduira par `new DrawLineCommand(3, 0, -1)`. Le premier paramètre indique le nombre de pas à faire, les deux autres paramètres la direction du déplacement (même colonne, ligne précédente)
3. **Ajouter une commande** qui change le caractère à dessiner (nécessite de modifier la classe `Robot`)

```
Command[] program = {
        moveTo(1, 5), change('-'), drawRight(3),
        change('|'), drawUp(2)
};
```

2 Description des classes

2.1 La classe `Robot`

Code fourni, voir annexe.

- Un `Robot` possède un écran (`Screen`) sur lequel il peut dessiner des caractères.
- La taille de l'écran est fixée à la construction.
- Le robot a une "position de dessin courante" définie par 2 variables.
- Il agit sur l'écran en envoyant un caractère (étoile) à une position donnée.

Méthodes publiques

- `drawChar` : met une étoile sur l'écran à la position courante ;
- `absoluteMove` : change la position courante (saute à la position indiquée) ;
- `relativeMove` : *décale* la position courante ;
- `execute` : affiche l'image construite en exécutant les "commandes" d'un programme.

Convention pour les coordonnées : origine en haut à gauche, numéro de colonne (de gauche à droite) suivi par numéro de ligne (du haut vers le bas).

2.2 La classe `Screen`

Code fourni.

- Un `Screen` construit une image dans un tableau de caractères.
- On y place les caractères un par un.
- Il sait s'afficher.

Méthodes publiques :

- `clear` : remplit l'image avec des espaces ;
- `putChar` : ajoute un caractère sur l'image ;
- `display` : affiche l'image.

2.3 Interface `Command`

À écrire.

Toute commande aura une méthode `execute` (celle qui est appelée par la méthode `Robot ::run`).

2.4 Classe `DrawRightCommand` (exemple de commande)

Code fourni, comme exemple de commande

- Le constructeur mémorise le nombre d'étoiles à dessiner vers la droite.
- La méthode `execute` ajoute les caractères un par un.

3 Annexes

3.1 Classe `Robot`

```
public class Robot {

    private final int nbCols, nbRows;
    private final Screen screen;
    private int currentCol, currentRow;  // position actuelle

    public Robot(int nbCols, int nbRows) {
        this.nbCols = nbCols;
        this.nbRows = nbRows;
        this.currentCol = 0;
        this.currentRow = 0;
        screen = new Screen(nbCols, nbRows);
    }

    public void run(Command[] program) {
        screen.clear();
        for (Command c : program) {
            c.execute(this);
        }
        screen.display();
    }

    public void drawChar() {
        screen.putChar('*', currentCol, currentRow);
    }

    public void relativeMove(int c, int r) {
        currentCol += c;
        currentRow += r;
    }

    public void absoluteMove(int c, int r) {
        currentCol = c;
        currentRow = r;
    }
}
```

3.2 Classe `Screen`

```
public class Screen {

    private final int nbCols, nbRows;
    private final char[][] image;

    public Screen(int nbCols, int nbRows) {
        this.nbCols = nbCols;
        this.nbRows = nbRows;
        this.image = new char[nbCols][nbRows];
```

```
    }

    public void clear() {
        for (int r = 0; r < nbRows; r++) {
            for (int c = 0; c < nbCols; c++) {
                image[c][r] = ' ';
            }
        }
    }

    public void putChar(char c, int col, int row) {
        if (0 <= row  0 && row < nbRows && 0 <= col && col < nbCols) {
            image[col][row] = c;
        }
    }

    public void display() {
        displayBar();
        for (int r = 0; r < nbRows; r++) {
            System.out.print("|");
            for (int c = 0; c < nbCols; c++) {
                System.out.print(image[c][r]);
            }
            System.out.println("|");
        }
        displayBar();
    }

    private void displayBar() {
        System.out.print("+");
        for (int c = 0; c < nbCols; c++) {
            System.out.print("-");
        }
        System.out.println("+");

}
```

3.3 Classe `DrawRightCommand`

```
public class DrawRightCommand implements Command {
    private final int nb;

    public DrawRightCommand(int nb) {
        this.nb = nb;
    }

    @Override
    public void execute(Robot robot) {
        robot.drawChar();
        for (int i = 0; i < nb; i++) {
            robot.relativeMove(1,0);
            robot.drawChar();
        }
    }
}
```